

Decremento/divisão/transformação e conquista

10.1 Decremento e conquista

O projeto por decremento e conquista baseia-se no relacionamento entre a solução de uma instância e a solução de uma **sub-instância menor do problema**. Ou seja, utilizamos a solução da instância menor para obter a solução da instância maior (**conquista**). O tamanho da instância é **reduzido por uma mesma constante** em cada iteração. Geralmente esta constante vale 1, ou seja, em cada iteração reduzimos em 1 o tamanho da instância.

A forma natural de projetar por decremento e conquista é utilizando **recursão**, mas vimos que isto pode provocar **estouro de pilha**. Felizmente é geralmente é simples **traduzir** o algoritmo recursivo produzido **para uma versão iterativa**.

Por exemplo, para calcular b^n onde n é inteiro não negativo e b é não nulo, podemos utilizar o fato de que $b^0 = 1$ e $b^n = b^{n-1} \cdot b$. Ou seja, para resolver a instância (b, n) (que tem tamanho n) utilizamos a solução da instância $(b, n - 1)$ (que tem tamanho $n - 1$). Como o restante do algoritmo tem tempo constante, a complexidade é dada pela recorrência $T(n) = T(n - 1) + \Theta(1)$ que tem solução $\Theta(n)$. Note que o tamanho real da entrada é o número de bits para representar b e n .

A instância pode também ser reduzida por um **fator constante**, ou seja, em cada iteração dividimos o tamanho da entrada por uma mesma constante. Esta constante geralmente vale 2 (divisão ao meio). Trataremos este caso na seção sobre **divisão e conquista**.

Podemos também ter **reduções de tamanho variável**. Por exemplo, no algoritmo de Euclides vimos que $\text{mdc}(m, n) = \text{mdc}(n, m \bmod n)$. Portanto, a instância de tamanho $m + n$ passou a ter tamanho $n + m \bmod n$. A instância diminuiu, pois $m < m \bmod n$, mas a redução não é constante nem é por fator constante.

10.1.1 Exemplo: ordenação por inserção

Vamos considerar o problema de ordenar uma lista $L[1..n]$ aplicando o decremento de 1 elemento e conquista. Retirando o último elemento podemos recursivamente ordenar a lista restante. Na etapa de conquista basta posicionar corretamente o último elemento na lista que foi ordenada recursivamente. Esta estratégia é chamada de ordenação por inserção. A complexidade é obtida pela recorrência $T(n) = T(n - 1) + O(n)$, que tem solução $O(n^2)$.

Algoritmo: ordenação_por_inserção_recursivo(L, n)

Entrada: Lista L com índices começando em 1. Inteiro positivo n.

Saída : Modificação de L para que seus n primeiros elementos fiquem em ordem não decrescente.

```
1 if n > 1 then // O(1)
2   ordenação_por_inserção_recursivo(L, n - 1) // T(n - 1)
   // Coloca L[n] na posição correta em L[1..n - 1]
3   (j, v) ← (n - 1, L[n]) // O(1)
4   while j ≥ 1 and L[j] > v do // O(n)
5     L[j + 1] ← L[j] // O(n)
6     j ← j - 1 // O(n)
7   L[j + 1] ← v // O(1)
```

Para evitar estouro de pilha temos que transformar o algoritmo recursivo no iterativo abaixo. Ao invés de realizar chamadas recursivas, acrescentamos um for para variar o tamanho i de 2 até n . O código restante é similar, pois apenas trocamos o n por i . A complexidade é dominada por $O(\sum_{i=2}^n i) = O(n^2)$, ou seja, tem a mesma complexidade da versão recursiva. Visualização da execução do algoritmo em <https://visualgo.net/pt/sorting>

Algoritmo: ordenação_por_inserção(L, n)

Entrada: Lista L com índices começando em 1. Inteiro positivo n.

Saída : Modificação de L para que seus n primeiros elementos fiquem em ordem não decrescente.

```
1 for i de 2 até n do // O(n)
   // Coloca L[i] na posição correta em L[1..i - 1]
2   (j, v) ← (i - 1, L[i]) // O(n)
3   while j ≥ 1 and L[j] > v do // O(∑_{i=2}^n i)
4     L[j + 1] ← L[j] // O(∑_{i=2}^n i)
5     j ← j - 1 // O(∑_{i=2}^n i)
6   L[j + 1] ← v // O(n)
```

10.2 Divisão e conquista

- O projeto de algoritmos por divisão e conquista consiste em
1. Dividir o problema em vários subproblemas do mesmo tipo e com aproximadamente o mesmo tamanho.
 2. Os subproblemas são resolvidos recursivamente, a menos que seja pequeno o bastante para permitir solução direta.
 3. As soluções dos subproblemas são combinadas para obter a solução do problema original (conquista).

Por exemplo, suponha que desejamos **somar os número na lista** $L[1..n]$ utilizando divisão e conquista. Podemos recursivamente somar os números nas duas metades $L[1..\lfloor n/2 \rfloor]$ e $L[\lfloor n/2 \rfloor + 1..n]$. Se s_1 é a soma retornada para a primeira metade, e s_2 é a soma retornada para a segunda metade, então a soma dos elementos da lista original $L[1..n]$ vale $s_1 + s_2$. Como resolvemos recursivamente duas sub-instâncias de tamanho aproximadamente $n/2$, e gastamos tempo constante nas outras linhas do algoritmo, a complexidade é obtida pela recorrência $T(n) = 2T(n/2) + \Theta(1)$ que tem solução $\Theta(n)$. Note que neste caso a divisão e conquista tem a mesma complexidade do algoritmo de força bruta que consiste que percorrer a lista somando os elementos.

Como vimos no exemplo, a divisão e conquista não necessariamente vai fornecer complexidade menor que um algoritmo força bruta, mas **para alguns problemas** teremos algoritmo de divisão e conquista **mais eficiente que o força bruta**. Além disso, a divisão e conquista permite utilizar **computação paralela**, pois cada subproblema pode ser resolvido em um processador distinto.

10.2.1 Exemplo: Mergesort

O algoritmo de ordenação Mergesort consiste em dividir a lista ao meio, recursivamente ordenar cada metade, e em seguida aplicar o procedimento Intercala para transformar as duas metades ordenadas na lista final ordenada.

O procedimento intercala cria uma cópia de cada uma das metades ordenadas, chamadas E e D . A cópia E é percorrida através do índice i , e a cópia D através do índice j . Os índices começam na primeira posição de cada cópia. Em cada iteração, o menor valor entre $E[i]$ e $D[j]$ é transferido para a lista L . Se $E[i]$ foi transferido para L , avançamos o índice i . Caso contrário, avançamos o índice j . O valor ∞ é acrescentado no final das cópias para garantir que quando uma das cópias terminar de ser percorrida, os elementos restantes da outra cópia serão transferido para a lista L .

Algoritmo: Mergesort(L, ini, fim)

Entrada: Lista L. Inteiros não negativos ini e fim, com ini ≤ fim.

Saída : Modificação de L para que seus elementos da posição ini até a posição fim fiquem em ordem não decrescente.

```
1 if ini < fim then
2   meio ← ⌊(ini + fim)/2⌋ // O(1)
3   Mergesort(L, ini, meio) // T(n/2)
4   Mergesort(L, meio + 1, fim) // T(n/2)
5   Intercala(L, ini, meio, fim) // O(n)
```

Algoritmo: Intercala(L , ini, meio, fim)		
Entrada: Lista L . Inteiros não negativos ini, meio e fim, com $ini \leq meio \leq fim$. Os elementos em $L[ini..meio]$ e $L[meio + 1..fim]$ estão em ordem não decrescente.		
Saída : Modificação de L para que seus elementos da posição ini até a posição fim fiquem em ordem não decrescente.		
1	$(E, D) \leftarrow (L[ini : meio], L[meio + 1 : fim])$	// $O(n)$
2	$E.append(\infty); D.append(\infty)$	// $O(1)$
3	$(i, j) \leftarrow (1, 1)$	// $O(1)$
4	for k de ini até fim do	// $O(n)$
5	if $E[i] \leq D[j]$ then	// $O(n)$
6	$L[k] \leftarrow E[i]$	// $O(n)$
7	$i \leftarrow i + 1$	// $O(n)$
8	else	
9	$L[k] \leftarrow D[j]$	// $O(n)$
10	$j \leftarrow j + 1$	// $O(n)$

Como o procedimento Intercala tem tempo linear, a recorrência que fornece o tempo de execução do Mergesort vale $T(n) = 2T(n/2) + O(n)$, e portanto a complexidade do algoritmo é $O(n \log n)$. Visualização da execução do algoritmo em <https://visualgo.net/pt/sorting>

10.3 Transformação e conquista

No projeto por transformação e conquista o problema é modificado para uma forma mais fácil de resolver, e em seguida é resolvido (conquista). As três principais variações são:

Simplificação de instância: transforma em uma instância mais simples ou mais conveniente do mesmo problema.

Mudança de representação: modifica a forma como a instância é representada.

Redução de problema: transforma em uma instância de outro problema para o qual já existe algoritmo disponível.

10.3.1 Exemplo de simplificação: pré-ordenar

A ordenação dos elementos em uma lista pode tornar o problema mais fácil de resolver. Como exemplo, considere o problema de **determinar se existem elementos repetidos** em uma lista com n elementos. Temos abaixo um algoritmo **força bruta que testa todos os pares de elementos**. A complexidade é dominada por $O(\sum_{i=1}^{n-1} (n-i)) = O(\sum_{k=1}^{n-1} k) = O(n^2)$.

Algoritmo: Tem_repetidos_força_bruta($L[1..n]$)		
Entrada: Lista $L[1..n]$.		
Saída : Booleano indicando se L tem elementos repetidos.		
1	for i de 1 até $n - 1$ do	// $O(n)$
2	for j de $i + 1$ até n do	// $O(\sum_{i=1}^{n-1} (n-i))$
3	if $L[i] = L[j]$ then	// $O(\sum_{i=1}^{n-1} (n-i))$
4	return true	// $O(1)$
5	return false	// $O(1)$

Se ordenarmos a lista teremos um problema mais simples, pois basta percorrer a lista verificando se existe um par de elementos consecutivos iguais, como indicado no algoritmo abaixo. Neste caso gastamos $\Theta(n \log n)$ para ordenar a lista, e $\Theta(n)$ para **percorrer lista ordenada** procurando um par de elementos repetidos. Logo, a **complexidade final será $\Theta(n \log n)$** , e portanto é mais eficiente que o força bruta.

Algoritmo: Tem_repetidos($L[1..n]$)		
Entrada: Lista $L[1..n]$.		
Saída : Booleano indicando se L tem elementos repetidos.		
1	$L.sort()$	// $O(n \log n)$
2	for i de 2 até n do	// $O(n)$
3	if $L[i] = L[i - 1]$ then	// $O(n)$
4	return true	// $O(1)$
5	return false	// $O(1)$

Pré-ordenar **não necessariamente vai produzir um algoritmo mais eficiente**. Por exemplo, para buscar um elemento em uma lista podemos ordená-la em tempo $\Theta(n \log n)$ e em seguida realizar uma busca binária em tempo $\Theta(\log n)$, resultando em um algoritmo de tempo $\Theta(n \log n)$. Por outro

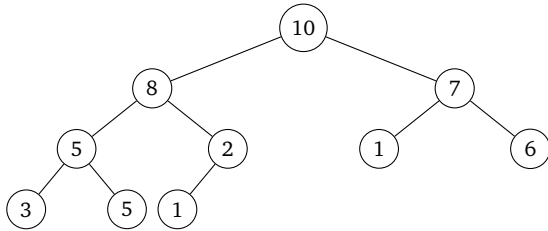
lado, um algoritmo força bruta realizaria uma busca sequencial percorrendo a lista em tempo $\Theta(n)$. Ou seja, o algoritmo força bruta é mais eficiente neste caso.

Muitos **algoritmo geométricos** são simplificados através da **ordenação dos pontos** usando algum critério, como uma das coordenadas, a distância até uma reta, ou algum ângulo. Alguns problema em **grafos direcionados acíclicos** são simplificados por uma **ordenação topológica**, como por exemplo os problemas de encontrar um caminho mais longo ou mais curto nestes grafos.

10.3.2 Exemplo de mudança de representação: heaps

Uma heap é uma árvore binária (cada nó tem no máximo 2 filhos) que atende as condições abaixo:

- Os nós são acrescentados um nível de cada vez, da esquerda para a direita. No exemplo abaixo, o próximo nó seria o filho à direita do 2.
- Se for uma min-heap, a chave de cada nó é menor ou igual às chaves dos filhos. Se for uma max-heap, a chave de cada nó é maior ou igual às chaves dos filhos. No exemplo abaixo temos uma max-heap.



Heaps podem ser representadas como arrays, onde a raiz é armazenada na posição 1. Para cada elemento na posição i , temos que:

- O filho à esquerda está na posição $2i$.
- O filho à direita está na posição $2i + 1$.
- O pai está na posição $\lfloor n/2 \rfloor$.

Por exemplo, a chave 7 está na posição 3, e portanto o filho à esquerda está na posição $2 \cdot 3 = 6$ (chave 1) e o filho à direita está na posição $2 \cdot 3 + 1 = 7$ (chave 6). A chave 2 está na posição 5, e portanto seu pai está na posição $\lfloor 5/2 \rfloor = 2$ (chave 8).

10	8	7	5	2	1	6	3	5	1
1	2	3	4	5	6	7	8	9	10

A representação como array é mais simples e dispensa o gerenciamento de ponteiros para os filhos. Da mesma forma, podemos transformar um array em uma heap, bastando apenas reposicionar os elementos para que as chaves satisfaçam a propriedade de min-heap/max-heap. Como exemplo desta transformação temos o algoritmo de ordenação Heapsort. Visualização da execução do Heapsort em <https://visualgo.net/en/heap>

Algoritmo: Heapsort($L[1..n]$)		
Entrada: Lista $L[1..n]$.		
Saída : Modificação de L para que seus elementos fiquem em ordem não decrescente.		
1	inverte o sinal das chaves dos elementos em L (max-heap)	// $O(n)$
2	heapify(L)	// $O(n)$
3	for i de n até 1 do	// $O(n)$
4	$L[i] \leftarrow \text{heappop}(L)$	// $O(n \log n)$

10.3.3 Exemplo de redução: para programação linear

O problema de **programação linear** consiste em maximizar ou minimizar uma **função objetivo linear** sujeita a várias **restrições** na forma de equações ou inequações lineares. Por exemplo,

$$\begin{aligned} \min \quad & 2x_1 + 3x_2 - 5x_3 \\ \text{sujeito a} \quad & x_1 - 3x_2 = 12 \\ & x_2 + 4x_3 \leq 8 \end{aligned}$$

Uma solução consiste em encontrar valores para as variáveis x de modo a maximizar ou minimizar a função objetivo, respeitando todas as restrições. **Existe algoritmo de tempo polinomial** para a encontrar uma solução ótima para um problema de programação linear. Na prática o **método simplex é o mais rápido**, mesmo tendo complexidade de pior caso exponencial. Observe que a solução pode ter valores fracionários. Se acrescentarmos a restrição de que a solução tem alguns **valores inteiros**, **não temos a garantia de resolver em tempo polinomial**.

Muitos problemas podem ser transformados em um problema de programação linear, permitindo usar o simplex para resolvê-lo. Como exemplo

considere o **problema da mochila fracionário**, que recebe como entrada um tamanho da mochila W , um conjunto de itens S , e o peso $p[i]$ e valor $v[i]$ de cada item $i \in S$. O objetivo é escolher um subconjunto dos itens de valor máximo, com a restrição de que a soma dos pesos dos itens selecionados é no máximo o tamanho da mochila W . Podemos modelar como um problema de programação linear da forma indicada abaixo:

max

$$\sum_{i \in S} v[i] \cdot x_i$$

sujeito a

$$\sum_{i \in S} p[i] \cdot x_i \leq W$$

$$0 \leq x_i \leq 1$$

$$\forall i \in S$$

Cada variável x_i fornece a proporção do item i que será usada na solução, então observe que estamos assumindo que os itens podem ser fracionados. Portanto, para resolver o problema basta executar o método simplex. Se alguns itens não puderem ser fracionados (acrescenta ou não o item), temos a restrição adicional de que algumas variáveis são inteiras, impedindo assim o uso do método simplex. Neste caso temos um **problema da mochila binário**.

Veremos **outros exemplos de redução na aula sobre NP-completude**, pois para mostrar que nosso problema é difícil basta mostrar que um problema difícil qualquer se reduz ao nosso, ou seja, que podemos usar um algoritmo para o nosso problema para resolver uma classe de problemas em que nenhum algoritmo eficiente é conhecido.

Leitura complementar

- Divisão e conquista: capítulo 4 do Cormen, capítulo 2 do Vazirani, seção 4.10 do Skiena.
- Ordenação por inserção: seção 2.1 do Cormen, introdução do capítulo 1 do Skiena.
- Heaps: capítulo 6 do Cormen, seção 4.5.2 do Vazirani, seção 4.3 do Skiena.
- Mergesort: seção 2.3.1 do Cormen, seção 2.3 do Vazirani, seção 4.5 do Skiena.
- Programação linear: capítulo 29 do Cormen, capítulo 7 do Vazirani, seção 13.6 do Skiena.